Chapter 09 • Conclusion

Course Final Project

16 pages

elevify

Course Final Project

16 pages

Course Completion Project

Driving Question

Build a functional personal journal app using Flet that allows creating, viewing, editing, and deleting journal entries, with data persistence via SQLite and a polished dark/light theme toggle.

Production Format

Argumentative essay — 800 to 1500 words.

Context and Provocation

Objective

Build a complete multi-page personal journal application using Flet. This project integrates everything you've learned: core controls, layout, state management, navigation, dialogs, async data persistence with SQLite, theming, and a final desktop build. You will produce a working app that can create, view, edit, and delete journal entries, store them in a local database, support light/dark theme toggling, and be packaged as a standalone executable.

Creative Brief

Your friend wants a simple digital diary to jot down daily thoughts. “I need something that feels good to use—easy to add entries, see them in a nice list, read them, change them, and delete the ones I don’t want. And can you make it look modern? I love dark mode.” They’ll run it on their Windows laptop. This is a real request—not a homework assignment. Your job is to deliver a polished, user-friendly app that they can actually use. You have full creative freedom on colors, fonts, and layout. Show off your skills!

Objective

Build a complete multi-page personal journal application using Flet. This project integrates everything you've learned: core controls, layout, state management, navigation, dialogs, async data persistence with SQLite, theming, and a final desktop build. You will produce a working app that can create, view, edit, and delete journal entries, store them in a local database, support light/dark theme toggling, and be packaged as a standalone executable.

Creative Brief

Your friend wants a simple digital diary to jot down daily thoughts. “I need something that feels good to use—easy to add entries, see them in a nice list, read them, change them, and delete the ones I don’t want. And can you make it look modern? I love dark mode.” They’ll run it on their Windows laptop. This is a real request—not a homework assignment. Your job is to deliver a polished, user-friendly app that they can actually use. You have full creative freedom on colors, fonts, and layout. Show off your skills!

Materials and Resources

- Python 3.8+ and Flet installed
- A code editor (VS Code recommended)
- SQLite3 (included with Python)
- Optional: httpx if you want to add a quote of the day feature in bonus
- Access to the Flet documentation (flet.dev)

Execution Steps

1. Set up your project structure and initialize the database

- Create a file 'database.py' with functions to create the SQLite database and table (columns: id, title, content, date).
- Professional insider tip: Use 'sqlite3.connect' with 'check_same_thread=False' if you plan to use async handlers, and always close the connection.
- Common mistake to avoid: Forgetting to commit after INSERT/UPDATE/DELETE.

2. Design the main layout with a NavigationRail and a content area

- Use a 'Row' with a 'NavigationRail' on the left and a 'Column' on the right for views.
- Add “Home” and “Settings” destinations. Home shows the journal entry list; Settings will have a theme toggle.
- Professional insider tip: Use 'expand' on the content column to fill remaining space.
- Common mistake to avoid: Overcomplicating the layout; keep it simple with a single row.

3. Implement state management with a Python class

- Create a 'JournalApp' class that holds the page reference, current route, entry list, and theme mode.
- Use methods to switch views (home, add, edit, settings) by updating 'page.views' or using a conditional display approach.
- Professional insider tip: Store the database connection in the class so all methods can access it.
- Common mistake to avoid: Using global variables instead of encapsulating state in the class.

4. Build the entry list view with dynamic controls

• Show a 'ListView' of cards, each with title, date, and a delete button.

• Implement an async 'load_entries()' that queries the database and rebuilds the list.

• Professional insider tip: Use 'page.update()' after modifying the controls list.

• Common mistake to avoid: Not clearing the list before reloading, causing duplicates.

5. Add entry creation and editing with dialogs and navigation

• Create a separate view for adding/editing: use a 'View' with 'TextField' for title and content, and a save button.

• Use 'AlertDialog' for delete confirmation.

• Professional insider tip: For editing, pass the entry id as a route parameter and load existing data.

• Common mistake to avoid: Not handling the case when the user cancels editing.

6. Implement theming: light/dark toggle in settings

• Add a 'Switch' control in the settings view that toggles 'page.theme_mode' between 'ThemeMode.LIGHT' and 'ThemeMode.DARK'.

• Save the preference in a simple JSON file or SQLite settings table.

• Professional insider tip: Use 'page.update()' after changing the theme to see it instantly.

• Common mistake to avoid: Not persisting the theme preference; the app should remember it after restart.

7. Test the app thoroughly and package as an executable (optional, but recommended)

• Run 'flet run' to test. Check all CRUD operations, navigation, and theme toggling.

• Use 'flet build windows' (or your OS) to create a standalone executable.

• Professional insider tip: Add an icon and app metadata for a professional look.

• Common mistake to avoid: Forgetting to include assets (like fonts) in the build.

Self-Assessment Checklist

• The app starts and shows a list of journal entries (initially empty).

• You can add a new entry with a title, content, and it appears in the list with the current date.

• Clicking an entry opens a detail/edit view with existing data pre-filled.

• Editing and saving updates the entry in the database and the list.

• Deleting an entry shows a confirmation dialog, and the entry is removed.

• The dark/light theme toggle works and persists after restart.

• Navigation between Home and Settings is smooth (via NavigationRail or bottom nav).

• The app runs without errors and feels polished (consistent spacing, colors, etc.).

Bonus Challenge

- Add a search bar to filter entries by title.
- Fetch a daily motivational quote from an API (e.g., quotable.io) and display it on the home screen.
- Export the journal to a plain text or PDF file using 'flet"s file picker.

Submission Format

Accepted formats:

- video (.mp4/.mov/.webm)
- text (.pdf/.docx/.txt)

Minimum delivery:

- a .mp4 video (max 3 min) demonstrating the app’s main features (add, list, edit, delete, theme toggle)
- a .txt file with the complete Python source code (the main script, plus any helper modules) If you also completed the bonus challenge, include brief notes in the .txt file.

Development Guide

1. Research Phase
Look for examples of small apartment designs on sites like ArchDaily or Dezeen. Read news articles on micro-apartments to see both praise and criticism. Note how different designs handle the space-saving vs. comfort tradeoff. Use the course chapters on apartment planning and 3D modeling as your main guides.
2. Organization Phase
Start by stating your main argument — your position on balancing efficiency and comfort. Then outline three to four key points that support your view, using the mandatory references. For each point, plan what evidence or examples you'll use. Also think about one counterargument and how you'll respond to it.
3. Writing Phase
Write an introduction that presents the problem and your thesis. In the body paragraphs, each should focus on one key point — explaining a course concept and applying it to your design example. Use simple language: imagine explaining to a friend. Include specific examples, like how you'd place a bed or choose lighting. End with a conclusion that summarizes your argument and suggests practical tips for designers.
4. Review Phase
Check that your essay clearly states your position. Verify that you've used all four mandatory references correctly. Read it aloud to see if the logic flows smoothly. Ask yourself: would a beginner understand this? Remove any jargon or complex terms without explanation.

Self-Assessment Rubric

1. Database Persistence & CRUD

• Insufficient — CRUD operations missing or broken; data does not persist; no database used.

• Adequate — Basic CRUD works but may have minor issues (e.g., refresh needed, date not auto-filled); data persists but with occasional errors.

• Excellent — All CRUD operations work flawlessly; data persists across app restarts; SQLite is used with proper async handling; no data loss or duplication.

2. UI & Navigation

• Insufficient — Navigation broken or missing; no card-based list; add/edit views absent or non-functional; no delete dialog.

• Adequate — Navigation works but may be sluggish; cards present but styling inconsistent; add/edit views functional but missing some polish (e.g., no cancel button).

• Excellent — NavigationRail with Home and Settings; views switch instantly; entry list with cards (title, date, delete); add/edit views use TextFields; delete dialog; all layouts use expand correctly; polished styling.

3. Theming

• Insufficient — No theme toggle, or toggle does not change anything.

• Adequate — Toggle works but may not persist after restart; adaptation of all elements is incomplete (e.g., some text remains dark on dark).

• Excellent — Light/dark toggle works instantly; switch in Settings; theme persists after restart via file/database; all UI elements adapt properly.

4. Async & State Management

• Insufficient — No async; UI freezes on database operations; state managed with globals or not at all.

• Adequate — Some async usage but may have blocking calls occasionally; state management exists but may be partially global.

• Excellent — All database operations are async; UI remains responsive during database calls; app state encapsulated in a class with page reference; no global variables.

5. Code Organization & Documentation

• Insufficient — Code messy, no separation of concerns; no comments; build fails or not attempted.

• Adequate — Code organized but all in one file; some comments but sparse; build may require manual steps.

• Excellent — Code split into at least two files (main.py, database.py); functions clearly named; comments explain key parts; no unused imports; executable builds successfully.

6. Bonus Features (optional)

• Insufficient — No bonus attempted, or bonus features completely broken.

• Adequate — Bonus attempted but not fully functional (e.g., search bar does not filter properly).

• Excellent — At least one bonus feature implemented and demonstrated (search bar, quote of the day, or export). Works correctly, enhances the app.

Model Response & Assessment Reference

> *This guide presents possible paths, not single answers. An excellent essay may defend any of the theses below — or an original thesis not anticipated here — as long as it demonstrates argumentative rigor, correct use of sources, and autonomous critical reflection.*

1. Expected Thesis Position(s)

The central question admits multiple legitimate positions. Below are argumentative paths representing robust and distinct directions.

> ***Author note:*** *The recommended range is 2 to 4 theses. If the assignment has only one defensible answer, keep just Thesis A and adapt the wording. A third thesis often works best as a synthetic or dialectical position.*

1.1. Thesis A — {Short label}

{Full thesis statement, in one or two sentences. It should take a clear position on the central question.}

Main argumentative line: {One paragraph describing how this thesis sustains itself: the central claim, supporting reasoning, implicit framework, and what makes it defensible.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

1.2. Thesis B — {Short label}

{Full thesis statement — a meaningfully different position from Thesis A, not just a softer variant.}

Main argumentative line: {One paragraph describing the central claim, supporting reasoning, implicit framework, and what makes it defensible.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

1.3. Thesis C — {Synthetic/dialectical position, optional}

{Full thesis statement — a meaningfully different position from Thesis A, not just a softer variant.}

Main argumentative line: {Show explicitly how this position produces a new insight that neither A nor B alone can reach.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

2. Key Concepts and Their Correct Use

> ***Author note:*** *Include 3 to 5 key concepts per project. For each concept: precise definition + example of correct application + common pitfall. The pitfall section is critical — it tells the evaluator what to watch for.*

2.1. A — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work — not just being named, but actively grounding a claim.}

Common pitfall: {The most frequent way students misuse this concept: over-application, decorative citation, missing nuance, ignoring critiques, or conflating related concepts. Be concrete.}

2.2. B — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work and grounding a claim.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

2.3. C — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work and grounding a claim.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

3. Model Argumentative Structure

The outline below illustrates the structure of an excellent-level essay, regardless of which thesis the student chooses.

> ***Author note:*** *Word ranges are suggestions calibrated for a {total length}-word essay. Adjust proportionally if the assignment is shorter or longer.*

3.1. Introduction ({X}–{Y} words)

Contemporary hook: {Describe what kind of opening works for this project — a concrete example, striking data point, scene from culture, or current event. Adapt to the project's domain.}

Bridge to subject: {How the hook should pivot into the project's central question.}

Explicit thesis: The author's position in one or two clear, specific sentences, positioned directly against the central question.

3.2. Development ({X}–{Y} words)

Suggested: 2–3 paragraphs, each with a central argument.

Paragraph 1: {Function and how it advances the thesis.}

Paragraph 2: {Function and how it advances the thesis.}

Paragraph 3: {Function and how it advances the thesis.}

3.3. Counterargument ({X}–{Y} words)

An excellent essay does not merely mention an objection: it presents the strongest counterargument with intellectual honesty and responds consistently.

> ***Author note:*** *List the best counterargument for each thesis option, with a suggested response. The student should not feel that confronting the objection weakens their position — done well, it strengthens it.*

If thesis is A: {Strongest objection and possible response.}

If thesis is B: {Strongest objection and possible response.}

If thesis is C: {Strongest objection and possible response.}

3.4. Conclusion ({X}–{Y} words)

Restatement of thesis: Reaffirm the position in light of the full argumentation, without repeating the introduction verbatim.

Implications: {What this reflection reveals about something larger — the field, how we think, or a contemporary issue.}

Opening (no new information): A provocative question or forward-looking gesture that broadens the horizon without introducing arguments not developed in the essay.

3.5. Evidence and Data Types to Mobilize

- Specific passages from primary sources.
- Theoretical concepts with citation to the original work.
- Documented contemporary examples: news, official reports, data, or cultural productions.
- References to academic debates covered in the course.

4. Rubric — Excellent Level

Detailed description of what the evaluator should observe in an essay rated “Excellent” on each criterion.

> ***Author note:*** *The criteria below cover the standard dimensions of argumentative essays. Remove or adapt criteria that do not apply to the specific project.*

4.1. Thesis clarity

What is observed at the EXCELLENT level: The thesis appears in the introduction unequivocally. It is specific and directly engages the project's central tension. The entire essay is organized around it.

4.2. Use of course concepts

What is observed at the EXCELLENT level: Required references are defined precisely before application. Concepts are integrated organically into the argument, and their limits are recognized.

4.3. Quality of argumentation

What is observed at the EXCELLENT level: Each paragraph opens with a clear claim, sustains it with evidence, and connects back to the thesis. Logical progression is clear and the student analyzes rather than merely describes.

4.4. Counterarguments

What is observed at the EXCELLENT level: The student presents the strongest objection to their position and responds with a substantial argument. The thesis emerges stronger from confronting the critique.

4.5. Originality of analysis

What is observed at the EXCELLENT level: The essay goes beyond reproducing course materials. The student offers original connections, asks new questions, or proposes a novel interpretation from familiar evidence.

4.6. Connection to the contemporary

What is observed at the EXCELLENT level: The dialogue between subject and present is specific and grounded. Concrete contemporary examples illuminate both sides.

4.7. Structure and academic norms

What is observed at the EXCELLENT level: Cohesive structure with fluid transitions. Citations follow the established academic pattern, and references are complete and correctly formatted.

5. Common Argumentative Errors

The errors below are the most frequent in this type of project. The evaluator should watch for them.

> ***Author note:*** *Errors 2, 3, and 4 apply to almost any argumentative project. Errors 1 and 5 are project-specific slots where particular risks may be described.*

5.1. {Project-specific error}

How it appears in the text: {Verbatim-style example showing the error in action.}

How to correct: {Concrete, actionable advice on what to do differently.}

5.2. Strawman fallacy

How it appears in the text: The student presents the counterargument in its weakest, easiest-to-refute form instead of confronting the strongest version.

How to correct: Reformulate the counterargument as an intelligent interlocutor would present it, then respond to that reason.

5.3. Decorative use of concepts

How it appears in the text: The student names a concept but never applies it — it appears as a loose citation, disconnected from the argument.

How to correct: After citing a concept, explain exactly how it grounds the specific argument of that paragraph.

5.4. False equivalence

How it appears in the text: The student treats as equivalent two phenomena that operate in radically different contexts or functions.

How to correct: Make differences explicit before drawing parallels. Use precise comparative language. Comparison is not equation.

5.5. {Project-specific error}

How it appears in the text: {Example}

How to correct: {Correction}

> ***Final reminder:*** *The quality of reasoning is always more important than the position defended. An essay that defends an unpopular thesis with rigor outperforms one that defends a consensus thesis superficially.*